

Model Generation Scripts

ThreeDViewport ships with five Python scripts that generate every `.glb` file referenced from the included sample projects. They use `trimesh` for geometry and `PIL` for procedural texture maps, then export glTF 2.0 with embedded PBR materials.

Script	Purpose
<code>generate_models.py</code>	Build a single-mesh shape (cube, sphere, mobius, molecule, etc.) with a chosen colour palette and material preset. Interactive prompts.
<code>generate_character.py</code>	Build the rigged <code>character</code> robot with a 4-region colour scheme (body / head / arms / legs). Preserves the FK hierarchy needed for follow-camera and pose animation.
<code>generate_station.py</code>	Build a hexagonal space station from a scaled benzene molecule — hollow atom hulls with window/doorway cutouts, glass panes, and ceramic inner shells.
<code>generate_emissive.py</code>	Build the nine simple primitives as pure-emissive glow markers / effect props at a chosen colour + intensity.
<code>generate_all.py</code>	Batch driver. Calls the others to produce every colour × material combination of every shape, plus uniform robots, stations, and emissives — optionally plus two-tone robots. See the <code>generate_all.py</code> section for exact counts.

Setup

The scripts depend on `trimesh`, `pillow`, and `numpy`. They expect a venv at `/tmp/glb_env`. That path is ephemeral — recreate it after a reboot:

```
bash
python3 -m venv /tmp/glb_env
/tmp/glb_env/bin/pip install trimesh pillow numpy
```

All commands below are run via the venv's interpreter, e.g.:

```
bash
/tmp/glb_env/bin/python3 generate_all.py
```

You can also use any other Python that has the three packages installed.

Where the files go

All scripts write into the app's "Model Library" folder — `modelsPathSecondary` from `~/Library/Application Support/ThreeDViewport/settings.json` — so generated files land where the app already browses them, with no copy/move step. The shared `models_root()` helper in `generate_models.py` reads that setting (falling back to a `./Models/` folder next to the script if the settings file is missing/unreadable); every script imports or uses it.

Script	Layout under the Model Library root
<code>generate_models.py</code> · <code>generate_character.py</code> · <code>generate_station.py</code> · <code>generate_emissive.py</code>	flat (files directly in the root)
<code>generate_all.py</code>	per-shape subfolders (e.g. <code>.../Models/sphere/</code> , <code>.../Models/robot/</code>)

`generate_all.py` writes shapes into subfolders by shape name and robots into a `robot/` subfolder. The three `buckyball-*` shapes all land in a single `buckyball/` subfolder. (Both flat root files and subfolders are reachable from the app's "add a model" open panel, which starts in this folder.)

Shared colour and material palette

`generate_models.py` defines the colour palette and material list; `generate_character.py` inlines copies of the same tables so it stays runnable on its own. The two stay in sync by convention.

Greyscale tonal ranges (keys 1–6)

A greyscale texture is remapped into a sub-range. Useful when you want monochrome models that still react crisply to lighting.

Key	Name	Range
1	Dark	10 – 90
2	Dark-Mid	40 – 150
3	Mid	80 – 180
4	Mid-Bright	130 – 220
5	Bright	180 – 255
6	Wide	0 – 255 (full range)

Colour palettes (keys 7–14)

Key	Name
7	Warm Amber
8	Cool Steel
9	Forest
10	Rose
11	Sunset
12	Ocean
13	Cosmic
14	Copper

Suffix	What it does
(none)	Plain three-stop gradient.
c1	Replaces the dark stop with a single complementary colour — e.g. 11c1 is Sunset with a teal pool of shadow.
c2	Inserts two complementary stops (at greyscale values 0 and 96) before the palette's mid stop — a richer dark-to-complement blend.

Append **a** to any choice to lock it in for the remaining prompts (e.g. 11c2a). A bare **a** skips all remaining prompts and uses defaults.

Solid white (key **w**)

w produces a **flat pure-white** base (label **white**) — the shape's pattern is discarded. Because the app's **Base Color** material override *multiplies* the base-colour texture, a white model tints to a clean solid colour in the viewer. This is the basis of the white → Material-Overrides tint → glue into an envelope → **Export Glued Model** reuse workflow. White molecules are built **multi-part** (see the molecule note) so each atom/bond group can be tinted independently before export.

Material presets

Key	Name	Metallic	Roughness
p	Polished Metal	1.00	0.10
b	Brushed Metal	0.90	0.45
m	Matte Plastic	0.00	0.85
c	Ceramic	0.05	0.40

Key	Name	Metallic	Roughness
r	Rubber	0.00	0.95

m is the default when you press Enter.

Total combinations

31 colours × 5 materials = **155 variants per shape**.

The colour count is 6 greyscale ranges + 8 palettes × 3 variants (plain / c1 / c2) + 1 solid white = 31.

generate_models.py

```
bash
/tmp/glb_env/bin/python3 generate_models.py      # interactive
/tmp/glb_env/bin/python3 generate_models.py -y   # accept all defaults (greyscale + i
```

Builds every shape in **SHAPES** (currently 25) one at a time, prompting for a colour palette and a material preset before each shape. Filenames encode the choice, e.g. **sphere-sunset-c2-ceramic.glb**.

Shape catalog

Group	Shapes	Default texture
Primitives	cube · cylinder · cone · pyramid · sphere · torus · tetrahedron · octahedron · icosahedron · dodecahedron · hexprism · capsule · disc	mostly linear / radial / cylindrical gradients
Surfaces	mobius · star · hyperboloid · trefoil · helix	spiral, cells, concentric rings, diagonal stripes, wood grain
Buckyballs	buckyball-162 · buckyball-642 · buckyball-2562	icospheres at three subdivisions (162, 642, 2562 verts)
Molecules	water · methane · benzene · dna	radial gradient / marble / angular stripes / spiral

Each builder returns **(mesh, uv, gray_array)**. The greyscale texture is coloured by the chosen palette function, encoded as a PNG, and embedded as the mesh's **baseColorTexture**. UVs are

Molecule note

In `generate_models.py` 's standalone mode, molecules (water, methane, benzene, dna) get a single PBR material like every other shape. In `generate_all.py`, the `white`, `c1`, and `c2` variants of molecules instead use `MOLECULE_BUILDERS` to produce a multi-part scene with three solid-colour groups — `heavy` (O / C atoms), `hydrogen` (H atoms), `bonds` (cylinders) — coloured from the palette's bright / complementary / mid stops (the `white` variant makes all three groups pure white, so you can tint each one independently in the app before re-gluing and exporting). This makes molecule colourings read as chemistry rather than as a single textured blob. DNA is a special case: both backbone strands map to `heavy` and the base-pair rungs map to `hydrogen`; there is no `bonds` node, so the palette's mid / `comp2a` stop is unused for DNA.

The three parts are exported with `heavy` as the root mesh and `hydrogen` / `bonds` parented to it, mirroring how the robot is structured (`hips` as the root with everything else descending). On import, ThreeDViewport renames the root to the filename basename and the whole molecule appears as a single hierarchical object — selectable and movable as one unit in Model mode, exactly like the robot.

`generate_character.py`

```
bash
/tmp/glb_env/bin/python3 generate_character.py      # interactive
/tmp/glb_env/bin/python3 generate_character.py -y  # default greyscale
```

Builds a rigged robot as `character-{bodyColor}-{headColor}-{armColor}-{legColor}.glb`. The four colour prompts are:

Region	Parts
Body	hips · torso · chest_panel · neck
Head	head sphere · visor · eyes · ears · antenna
Arms + Hands	shoulder · upper arm · elbow · forearm · hand (× left/right)
Legs + Feet	upper leg · knee · lower leg · ankle · foot (× left/right)

FK hierarchy

Every part's node origin sits at the joint that controls it, so rotating a node in ThreeDViewport pivots around the anatomical joint (important for keyframe animation and for camera-follow on sub-parts).

```

hips                                     (world root)
├─ torso                               pivot: hip-spine joint    [0, 0.145, 0]
│   └─ chest_panel
│       └─ neck                       pivot: top of torso      [0, 0.60, 0]
│           └─ head                   pivot: top of neck      [0, 0.69, 0]
│               └─ visor, eye_L, eye_R
│                   └─ ear_L, ear_R
│                       └─ antenna_pole, antenna_ball
├─ shoulder_L/R                       pivot: shoulder joint    [±0.305, 0.565, 0]
│   └─ upper_arm_L/R                  (same pivot as shoulder)
│       └─ elbow_L/R                 pivot: elbow joint      [±0.305, 0.215, 0]
│           └─ forearm_L/R           (same pivot as elbow)
│               └─ hand_L/R          pivot: wrist            [±0.305, -0.025, 0]
├─ upper_leg_L/R                      pivot: hip-leg joint     [±0.135, -0.025, 0]
│   └─ knee_L/R                      pivot: knee joint       [±0.135, -0.325, 0]
│       └─ lower_leg_L/R             (same pivot as knee)
│           └─ ankle_L/R             pivot: ankle joint     [±0.135, -0.625, 0]
│               └─ foot_L/R          (same pivot as ankle)

```

The script prints the final scene graph after building so you can verify the parenting matches.

Per-part materials

Different parts of the robot use different material presets internally:

Preset	Used by
<code>_METAL</code> (matte plastic)	hips, torso, shoulders, upper arms, forearms, upper legs, lower legs
<code>_HEAD</code> (rubbery sphere)	head
<code>_GLASS</code> (low-roughness with thin gloss)	chest panel, visor
<code>_JOINT</code> (slightly metallic)	neck, ears, antenna pole, elbow, hand, knee, ankle
<code>_GLOW(...)</code> (emissive)	eye LEDs, antenna ball

These are constants inside the script — adjust them there if you want a different look for the robot.

generate_station.py

```

bash
/tmp/glb_env/bin/python3 generate_station.py    # interactive
/tmp/glb_env/bin/python3 generate_station.py -y # default greyscale

```

Builds a stylized hexagonal space station as `station-{heavy}-{hydrogen}-{bond}-{material}.glb`. Geometry is benzene scaled ~2x and laid flat in the XZ plane (Y is up in the viewer). Three colour prompts mirror benzene's molecule-scene split (Heavy / Hydrogen / Bonds), then one material prompt.

Scene graph

Node	Contents
<code>heavy</code> (root)	6 carbon hubs (icospheres with face-cluster cutouts)
<code>hydrogen</code> (child)	6 outer pods (icospheres with face-cluster cutouts)
<code>bonds</code> (child)	6 radial C–H bonds + 6 ring-segment C–C bonds (open tubes)
<code>glass</code> (child)	36 spherical-cap window panes filling the window cutouts (translucent metallic)
<code>heavy_inner</code> (child)	Inverted-winding copy of the 6 carbon hubs, slightly inset toward each atom's centre, ceramic-style material in the same colour as <code>heavy</code>
<code>hydrogen_inner</code> (child)	Same idea for the 6 hydrogen pods
<code>bonds_inner</code> (child)	Thinner inverted-winding copies of the bond tubes, ceramic-style material in the same colour as <code>bonds</code>

Hull cutouts

Cutouts are produced by deleting any face of an atom's icosphere whose centroid lies inside an angular cone around a cutout direction. The leftover mesh is an open shell with irregular polygonal holes. There are two kinds:

- **Windows** — the “see through” openings. Larger half-angles, placed in the angular gaps between bond directions.
- **Bond-entry doorways** — sized just larger than the bond cylinder's cross-section so the bond cylinder docks cleanly with the hull. Placed along each bond direction at that atom.

Atom	Cutouts	Window half-angle	Bond-entry half-angle	Window layout
Carbon hub	3 windows + 3 doorways	22°	22° (matches CC bond's $\arcsin(0.054/0.15) = 21.1^\circ$)	±60° outward (flank C–H) + 180° inward (between C–Cs)
Hydrogen pod	3 windows + 1 doorway	28°	25° (matches CH bond's $\arcsin(0.044/0.11) = 23.6^\circ$)	Outward radial + ±120° on the equator

On `icosphere(2)` a 22° cone removes about 12 faces, a 28° cone about 24. After all cutouts the C atom is 248 faces (was 320, lost $6 \times 12 = 72$); the H pod is ~236 faces.

The carbon hubs end up with **6 openings evenly spaced 60° apart around the equator** (alternating window / doorway), reading as 6-port hubs rather than spheres with a few windows.

Glass panes

The `glass` submesh fills the window cutouts (not the doorway cutouts — those stay open for the future robot-through-tube animation). `_icosphere_with_cutouts` returns both the opaque `hull` and a `glass` mesh built from *the very faces that were removed* — so each pane is a piece of the original icosphere, perfectly matching the sphere's curvature in the hole it fills.

The glass material is the same on every batch combination (it's not driven by the palette), so the windows read as a consistent piece of station hardware:

```
GLASS_COLOR      = (180, 220, 255)  # pale arctic blue
GLASS_OPACITY    = 0.30              # a bit more smoke than the first pass
GLASS_METALNESS  = 0.70              # stronger HDRI reflection
GLASS_ROUGHNESS  = 0.10              # smooth surface for tidy reflections
alphaMode = "BLEND", doubleSided = true
```

With `baseColorFactor.w < 1.0` baked in, the glass automatically routes to the transparent pipeline through `Renderer.swift`'s opacity-or-alpha check. Reflections come from `metallicFactor` interacting with the loaded HDRI environment — turn an HDRI on in the viewer and the panes will pick up its highlights. Because the glass material is `doubleSided`, the *inside* surfaces of the spherical-cap panes reflect the HDRI too, which is why HDR highlights also appear inside each atom when you look through a doorway.

Bonds as open tubes

`_open_bond(p0, p1, radius)` builds a cylinder between two points then strips the end-cap faces (faces whose normal aligns with the cylinder's axial direction). The endpoints are placed on the atom *surfaces*, not the atom centres — `p0 = atom_a_center + atom_a_radius × bond_dir`, `p1 = atom_b_center - atom_b_radius × bond_dir`. The result is an open tube that terminates at the matching doorway in each atom hull, and the visible bond is the *whole* cylinder, not just the small stub between the two atom surfaces.

Inner shells (ceramic-matched interior)

The viewer's renderer has cull mode `.none` and the fragment shader uses the stored vertex normal directly — no flip for back-faces. That means the BACK of a polished-metal hull face is shaded with a normal pointing away from the camera, which sprays unhelpful strong HDRI reflections inside the station. To give the interior a soft, colour-matched look without changing the outside, every opaque part has an inner-shell counterpart:

- Vertices scaled by `1 - 0.005` toward the part's local centre so the inner surface sits just inside the outer and there's no z-fighting.
- Material fixed at `metallicFactor = 0.00` , `roughnessFactor = 0.85` (the "Matte Plastic" preset in `generate_models.py`).
- Base colour copied from the matching outer mesh (`heavy_rgb` , `h_rgb` , `bond_rgb`).

When the camera is outside and looks through a window cutout, the closest geometry past the window is the inner shell's near side — ceramic look wins. When the camera is inside an atom, the inner shell's front-faces are oriented correctly toward the camera and lighting computes normally. The outer shell's user-selected material (polished metal, brushed, matte, ceramic, rubber) still drives the exterior.

`doubleSided` on all materials

All three materials (`heavy` , `hydrogen` , `bonds`) are exported with `doubleSided = true` . The atoms are hollow shells with holes; the bonds are open-ended tubes. Both kinds of geometry expose their inner surface to the camera somewhere, and the renderer needs to draw both sides for the geometry to look correct from arbitrary angles. (ThreeDViewport's renderer already runs with cull mode `.none` everywhere, so this is effectively a no-op for this viewer — but the GLBs render correctly in other tools too.)

Renderer.swift transparency routing

`Renderer.swift` was updated alongside this work so any material with `baseColorFactor.w < 1.0` or `opacity < 1.0` routes into the transparent pipeline. The station is fully opaque now, so this doesn't affect it directly — but the routing change remains useful for any future model that bakes alpha into a GLB. `VideoExporter.swift` mirrors the same routing so exported `.mov` frames match what the viewport renders.

`generate_emissive.py`

```
bash
/tmp/glb_env/bin/python3 generate_emissive.py      # interactive (prompts colour + intensity)
/tmp/glb_env/bin/python3 generate_emissive.py -y   # default red, intensity 6
```

Builds the nine simple primitives — `cube` , `cylinder` , `pyramid` , `sphere` , `torus` , `tetrahedron` , `octahedron` , `hexprism` , `capsule` — as pure-emissive objects for use as glow markers and effect props (e.g. inside the station). Geometry comes straight from the existing primitive builders in `generate_models.py` ; only the material differs.

Material

Field	Value	Effect
<code>baseColorFactor</code>	<code>(0, 0, 0, 1)</code>	No diffuse colour — no shading contribution
<code>metallicFactor</code>	<code>0</code>	No metallic / reflection
<code>roughnessFactor</code>	<code>1</code>	No specular highlight
<code>emissiveFactor</code>	<code>(r, g, b) × intensity</code>	Pure emission, intensity folded in

The intensity is **pre-multiplied** into `emissiveFactor` rather than being written through the `KHR_materials_emissive_strength` glTF extension. ThreeDViewport's `GLTFLoader.swift` (emissive block — `mat.emissiveFactor *= strength`) collapses both forms into the same internal `mat.emissiveFactor` , and `Shaders.metal` adds it to the output colour without clamping, so the in-engine result is identical and the GLB doesn't depend on extension support in other tools.

Intensity

Default is **6.0** (reads as a clearly lit object against a dimmed scene). Range clamped to `[0.1, 100.0]` so a typed value can't break the material. Values above ~10 will saturate without bloom but stay distinct against the HDRI background; lower the Lights panel intensities and Atmosphere IBL intensity to make any emissive read more strongly relative to ambient.

Filenames carry the intensity so a single shape/colour can have several variants:

```
emissive-{shape}-{colour}-x{intensity}.glb
```

generate_all.py

```
bash
/tmp/glb_env/bin/python3 generate_all.py           # shapes + robots + stations +
/tmp/glb_env/bin/python3 generate_all.py --two-tone # also generates 900 two-tone
/tmp/glb_env/bin/python3 generate_all.py --shapes-only
/tmp/glb_env/bin/python3 generate_all.py --robot-only
/tmp/glb_env/bin/python3 generate_all.py --station-only
/tmp/glb_env/bin/python3 generate_all.py --emissive-only
```

Non-interactive. Iterates every (shape × colour × material), every (robot × colour), every (station × colour × material), and every (emissive shape × colour) combination, writing into the app's Model Library folder (`modelsPathSecondary`). Progress is printed on a single rewriting line per group so the output stays terse.

Mode	File count
Default (shapes + uniform robots + stations + emissives)	$3,150 + 30 + 150 + 270 = 3,600$
<code>--shapes-only</code>	$21 \times 30 \times 5 = 3,150$
<code>--robot-only</code>	30
<code>--robot-only --two-tone</code>	$30 + 900 = 930$
<code>--station-only</code>	$30 \times 5 = 150$
<code>--emissive-only</code>	$9 \times 30 = 270$
Default + <code>--two-tone</code>	$3,150 + 930 + 150 + 270 = 4,500$

Two-tone robots use body+legs as colour A and head+arms as colour B, named `robot-{A}-{B}.glb`. The station has its own 3-colour split via `palette_molecule_colors()` (same mechanism as benzene's c1/c2 variants), so there is no separate two-tone flag for it — `c1` and `c2` variants of the eight palette colours give the distinct heavy/hydrogen/bond colour combinations automatically. Emissives use a fixed default intensity (6.0) for the batch and sample the variant-specific stop for `-c1` (comp1) and `-c2` (comp2b) so all three palette variants are visually distinct — run `generate_emissive.py` interactively to dial in a different intensity for a single colour family.

The shapes pass uses `palette_molecule_colors()` for molecule shapes when the palette is `c1` or `c2`, producing the multi-part atom/bond colour split described above. The station pass uses the same function for the same reason.

Adding a new shape

1. Write a builder in `generate_models.py` that returns `(mesh, uv, gray_array)` — `mesh` is a `trimesh.Trimesh`, `uv` is per-vertex `(N, 2)`, `gray_array` is a 2D `uint8` greyscale texture (typically `TEX_SIZE × TEX_SIZE`).
2. Append it to `SHAPES` as `(name, builder, "human-readable pattern name")`.
3. If the output should live in a shared subfolder (like the buckyballs do), add an entry to `SHAPE_OUTPUT_DIRS`.
4. Run `generate_all.py` — it picks the new shape up automatically.

For a multi-coloured molecule with `c1` / `c2` variants:

1. Add a `build_X_scene(heavy_rgb, h_rgb, bond_rgb, metalness, roughness)` function that returns a `trimesh.Scene` with three nodes named `heavy`, `hydrogen`, and `bonds`, each with a solid-colour PBR material.
2. Add `heavy` first (no `parent_node_name`), then add `hydrogen` and `bonds` with `parent_node_name="heavy"`. This makes `heavy` the single root mesh so ThreeDViewport imports

4. `generate_all.py` will route `c1` / `c2` palette variants through your scene builder while keeping plain palettes / greyscale on the single-mesh `builder()` path.